



**DEPT. OF COMPUTER SCIENCE
AND
ENGINEERING
VARENDRA UNIVERSITY**



CSE 434: Parallel Processing and Distributed System Lab

Lab Assignment

On

**Distributed Attendance Management System using Java RMI
& Multithreading**

Submitted by:	Submitted to:
Name: I. S. M Rahik Session: Spring - 2026 Semester: 8 th ID: 223311207 Batch: 31 st Program: B.Sc. in CSE, Varendra University	<u>Course Teachers:</u> Md. Taufiq Khan Lecturer Dept. of CSE, Varendra University Md. Shahid Ahammed Shakil Lecturer Dept. of CSE, Varendra University

Date of Submission: 4/07/2026

Signature of Course Teacher: _____

1. Introduction

This project implements a distributed attendance management system in which multiple faculty members, each running an independent client application, can remotely mark and view student attendance stored on a single central server. The system was built with Java RMI (Remote Method Invocation) for network communication and Java Swing for the graphical user interfaces.

1.1 How RMI was implemented

The RMI architecture follows the standard three-part structure required by the assignment:

- Remote Interface (`common.AttendanceService`): extends `java.rmi.Remote` and declares the four remote services — `markAttendance()`, `getAttendanceHistory()`, `getAttendancePercentage()` and `getAllStudentRecords()`. Every method declares `RemoteException`.
- Remote Implementation (`server.AttendanceServiceImpl`): extends `UnicastRemoteObject`, so a dynamic proxy stub is generated automatically at export time (the legacy `rmic` compiler is not needed — modern RMI practice).
- Registry: the server creates its own registry with `LocateRegistry.createRegistry(1099)` and binds the remote object under the well-known name "AttendanceService" using `rebind()`.
- Client lookup: each client calls `LocateRegistry.getRegistry(host, 1099)` followed by `lookup("AttendanceService")` and receives the stub, after which every method call is transparently forwarded to the server over TCP.

Method parameters and results (the `AttendanceRecord` data model) travel across the network through Java serialization, so `AttendanceRecord` implements `Serializable`.

1.2 How multithreading was implemented

The RMI runtime automatically dispatches every incoming remote call on its own server-side worker thread, so when several faculty clients call the server simultaneously, their requests genuinely execute in parallel. All records are held in one shared in-memory `ConcurrentHashMap` keyed by the unique Student ID. Updates use `ConcurrentHashMap.compute()`, which performs an atomic create-or-update for one key, and the `AttendanceRecord` class additionally synchronizes access to its internal entry list. The server dashboard prints the name of the RMI worker thread that served each request, which makes the concurrent dispatch directly visible during the demonstration.

2. System Architecture and Thread Safety

The application is divided into three packages with a clear separation of concerns: `common` (shared contract and data model), `server` (business logic, storage, dashboard) and `client` (connection bootstrap and Swing GUI).

AttendanceRMI/	
— <code>src/common/AttendanceService.java</code>	Remote interface (RMI contract)
— <code>src/common/AttendanceRecord.java</code>	Serializable data model
— <code>src/server/AttendanceServiceImpl.java</code>	Remote object, thread-safe logic
— <code>src/server/AttendanceServer.java</code>	Registry creation + binding
— <code>src/server/ServerDashboard.java</code>	Swing live-request console
— <code>src/client/AttendanceClient.java</code>	Registry lookup + GUI launch
— <code>src/client/ClientGUI.java</code>	Swing GUI with four tabs
— <code>src/test/ConcurrencyTest.java</code>	Headless thread-safety stress test

2.1 Thread-safety mechanisms

Mechanism	Location in code	Purpose
RMI worker threads	RMI runtime (automatic)	Each incoming client call is dispatched on its own server thread
ConcurrentHashMap	AttendanceServiceImpl.records	Thread-safe shared in-memory store keyed by Student ID
compute() atomic update	markAttendance()	Create-or-update of one student's record is a single atomic action; no lost updates
synchronized methods	AttendanceRecord	Entry list and counters are read/written under the record's own lock
AtomicLong	requestCount	Lock-free counting of total served requests
Deep copies (copy())	All getter methods	Clients receive snapshots; server state is never aliased
SwingWorker + invokeLater	ClientGUI, ServerDashboard	Network calls off the EDT; all UI updates on the EDT

No raw unsynchronized HashMap is accessed anywhere in the code. Because compute() locks only the map bin of the affected Student ID, updates to different students proceed fully in parallel, giving both correctness and scalability.

3. Key Code Excerpt — Atomic, Thread-Safe Update

```
// server/AttendanceServiceImpl.java
private final ConcurrentHashMap<String, AttendanceRecord> records =
    new ConcurrentHashMap<>();

public String markAttendance(String studentId, String status)
    throws RemoteException {
    String id = validateId(studentId);
    String normalized = normalizeStatus(status);
    AttendanceRecord updated = records.compute(id, (key, existing) -> {
        AttendanceRecord r = (existing != null)
            ? existing : new AttendanceRecord(key);
        r.addEntry(normalized); // synchronized inside the record
        return r;
    });
    ...
}
```

The complete, fully commented source code of every file is included in the submitted zip archive.

4. User Interface and Output Screenshots

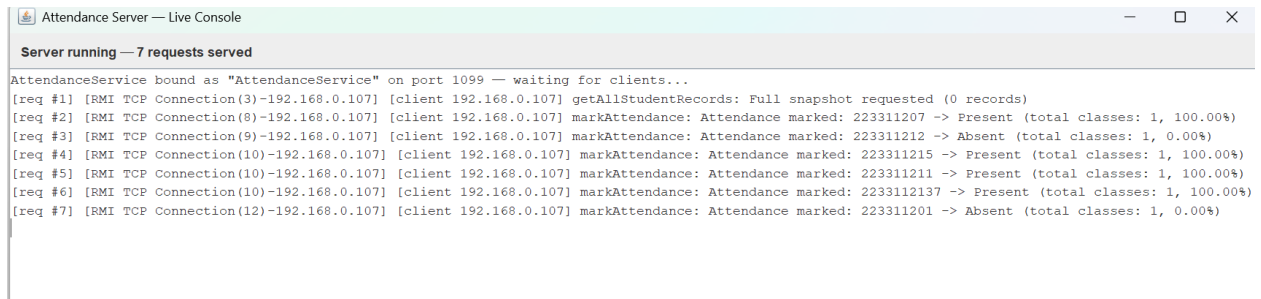


Figure 1: Server dashboard immediately after startup, showing the service bound in the RMI registry.

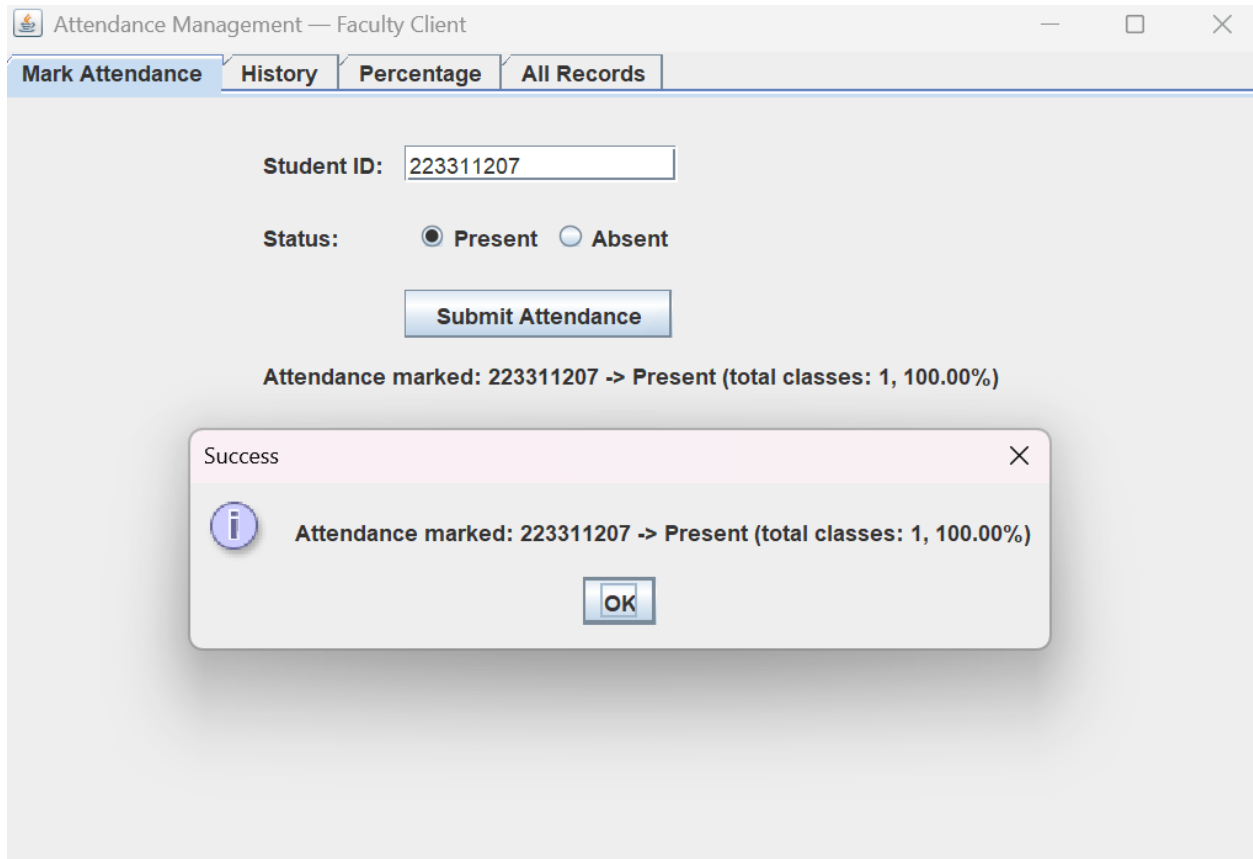


Figure 2: Client GUI — "Mark Attendance" tab with success dialog after marking a student Present.

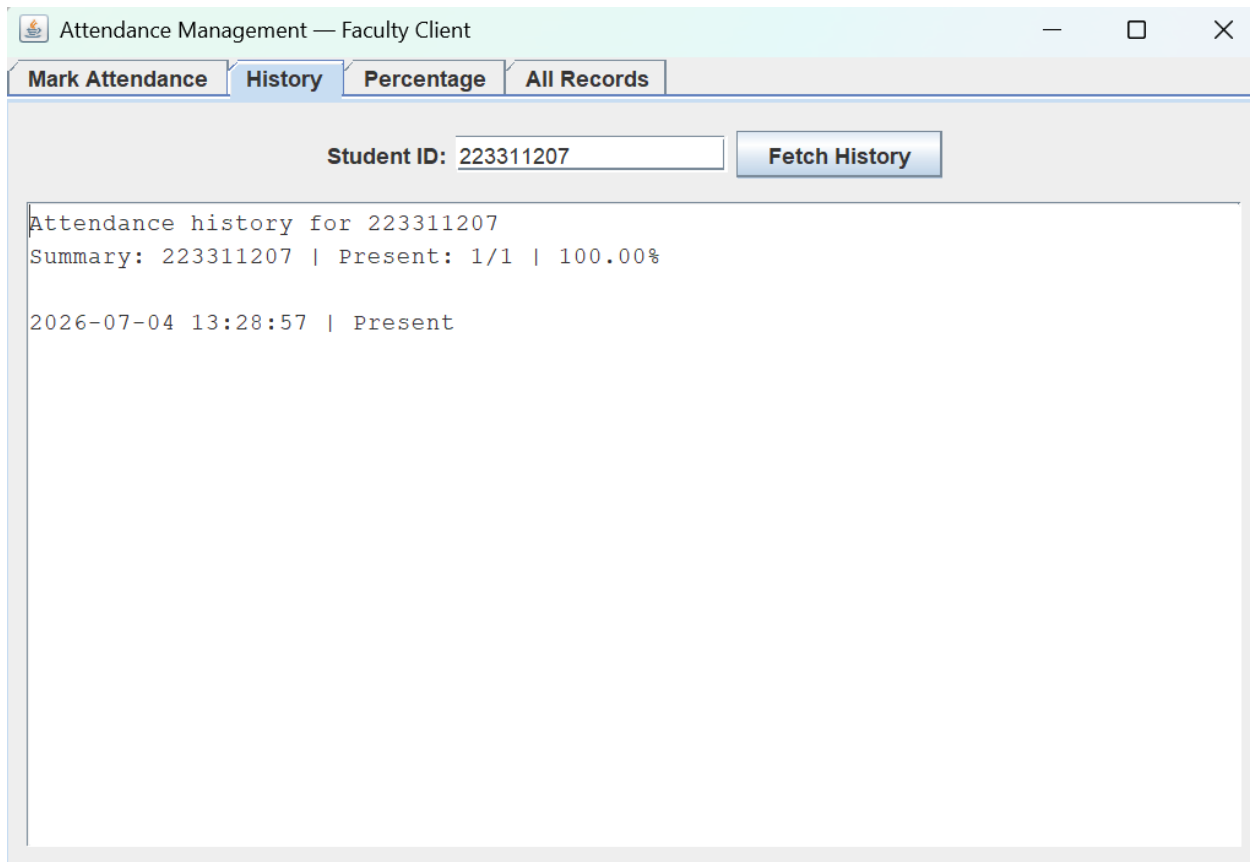


Figure 3: Client GUI — “History” tab showing the timestamped attendance log of one student

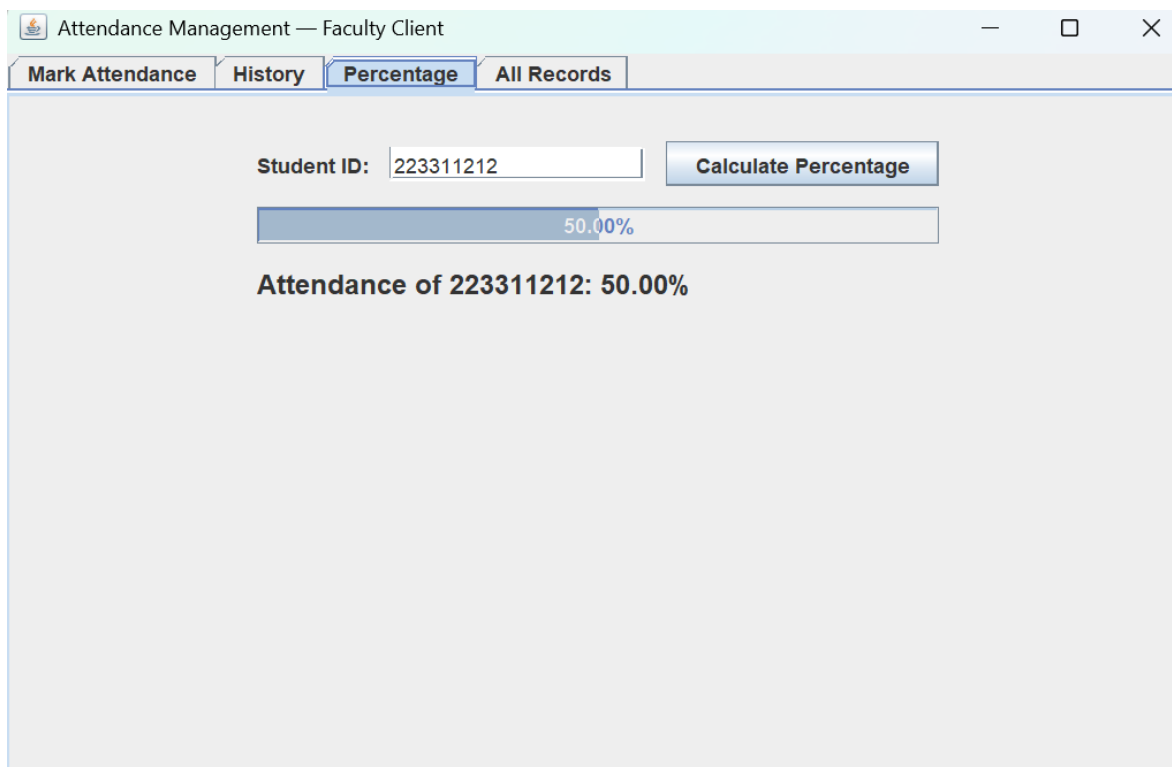


Figure 4: Client GUI — “Percentage” tab with the computed attendance percentage and progress bar.

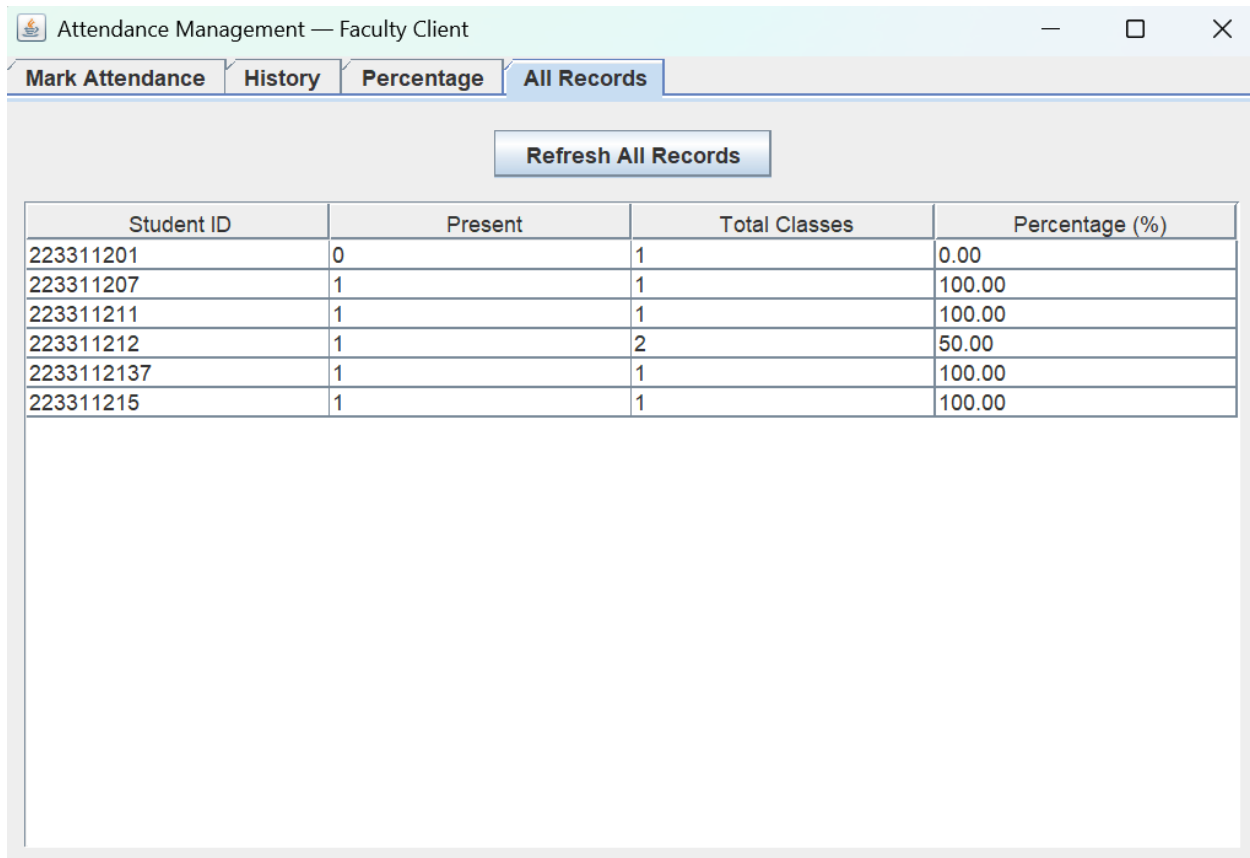


Figure 5: Client GUI — “All Records” tab listing every student stored on the server.

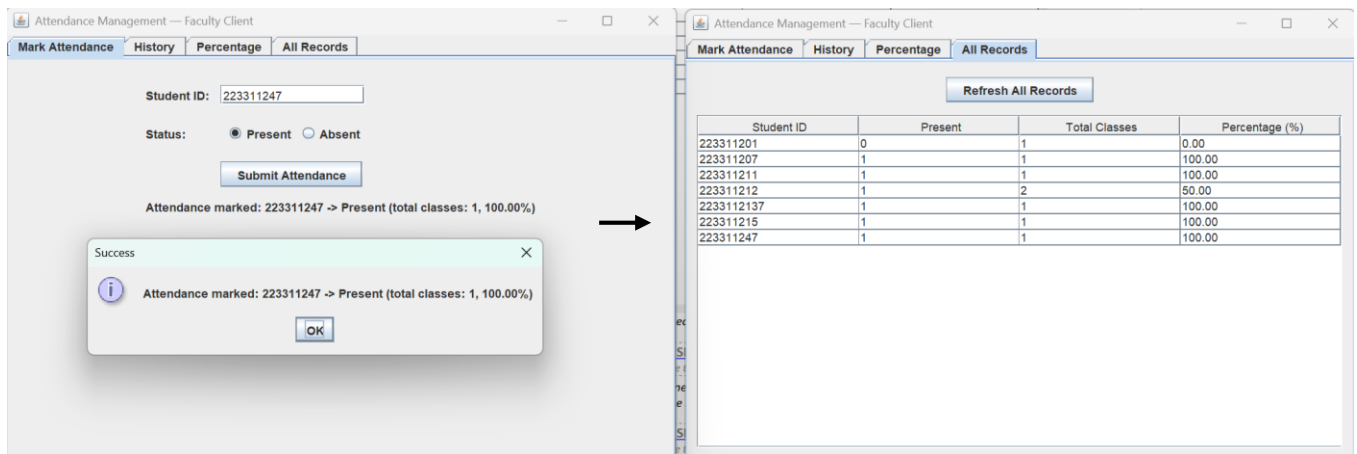


Figure 6: Two client windows open simultaneously; the record updated by Client 1 is instantly visible to Client 2, proving the shared server-side data structure.

```
Attendance Server — Live Console
Server running — 2015 requests served
[req #1990] [RMI TCP Connection(39)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311212 -> Present (total classes: 199, 49.25%)
[req #1988] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311215 -> Absent (total classes: 195, 49.23%)
[req #1995] [RMI TCP Connection(42)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Absent (total classes: 193, 50.78%)
[req #1992] [RMI TCP Connection(40)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311215 -> Present (total classes: 196, 49.49%)
[req #1993] [RMI TCP Connection(37)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Absent (total classes: 192, 51.04%)
[req #1996] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Present (total classes: 194, 51.03%)
[req #1994] [RMI TCP Connection(38)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311215 -> Absent (total classes: 197, 49.24%)
[req #1997] [RMI TCP Connection(39)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311247 -> Absent (total classes: 199, 50.75%)
[req #1998] [RMI TCP Connection(41)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311247 -> Absent (total classes: 199, 50.75%)
[req #1999] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311207 -> Absent (total classes: 200, 50.50%)
[req #2000] [RMI TCP Connection(40)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Absent (total classes: 195, 50.77%)
[req #2001] [RMI TCP Connection(38)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Present (total classes: 196, 51.02%)
[req #2003] [RMI TCP Connection(39)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311215 -> Present (total classes: 199, 49.75%)
[req #2002] [RMI TCP Connection(41)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311215 -> Present (total classes: 198, 49.49%)
[req #2004] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311212 -> Present (total classes: 201, 49.75%)
[req #2005] [RMI TCP Connection(39)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Absent (total classes: 197, 50.76%)
[req #2006] [RMI TCP Connection(41)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Absent (total classes: 198, 50.51%)
[req #2007] [RMI TCP Connection(38)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311207 -> Absent (total classes: 201, 50.25%)
[req #2008] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311247 -> Absent (total classes: 200, 50.50%)
[req #2009] [RMI TCP Connection(38)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311212 -> Present (total classes: 202, 50.00%)
[req #2010] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311215 -> Present (total classes: 200, 50.00%)
[req #2012] [RMI TCP Connection(38)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Absent (total classes: 199, 50.25%)
[req #2011] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311247 -> Absent (total classes: 201, 50.25%)
[req #2013] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311215 -> Present (total classes: 201, 50.25%)
[req #2014] [RMI TCP Connection(45)-192.168.0.107] [client 192.168.0.107] markAttendance: Attendance marked: 223311237 -> Absent (total classes: 200, 50.00%)
```

Figure 7: Server dashboard under concurrent load — note the different RMI worker-thread names serving the requests.

```
S-1001 -> total=200 (expected 200), present=100 (expected 100), 50.00% : OK
S-1002 -> total=200 (expected 200), present=100 (expected 100), 50.00% : OK
S-1003 -> total=200 (expected 200), present=100 (expected 100), 50.00% : OK
S-1004 -> total=200 (expected 200), present=100 (expected 100), 50.00% : OK
S-1005 -> total=200 (expected 200), present=100 (expected 100), 50.00% : OK
```

Figure 8: Output of test.ConcurrencyTest — 1000 concurrent remote calls, all counts exact, RESULT: PASS.

5. Testing Methodology

- Functional testing: every remote service was exercised through the GUI — marking Present/Absent, fetching history, computing percentages and listing all records — including invalid inputs (empty ID, unknown student), which produce clear error dialogs instead of crashes.
- Multi-client testing: three client instances were launched simultaneously; an update from any client became immediately visible to all others, confirming a single shared server-side data structure.
- Concurrency stress testing: test.ConcurrencyTest simulates 10 clients issuing 1000 concurrent markAttendance calls against 5 shared Student IDs. The test passes only if every final counter is exact (200 total, 100 Present, 50.00% per student). The test passed, empirically proving that no updates are lost under contention.
- Compilation quality: the project compiles with `javac --release 11 -Xlint:all` with zero warnings.

6. Conclusion

The project fulfils every requirement of the assignment: a proper RMI architecture (remote interface, implementation, registry, client lookup), a shared thread-safe in-memory data structure on the server, unique Student ID based records, all four required remote services, and user-friendly Swing GUIs for both the clients and the server. Concurrency is handled with `ConcurrentHashMap` atomic operations and per-record synchronization, and correctness under simultaneous multi-client load was verified with an automated stress test. Through this work the distributed-system implementation outcome (CO3) was achieved using modern Java tooling (PO6), and the design was critically evaluated and validated through systematic testing (K5).